AUTOBOTs: Autonomous Security Engineer

Project Requirement Specifications

Date: 19/11/2025

Group Members:

Syed Ahmed Haseeb	**2022557**
Muhammad Bilal	**2022359**
Muhammad Bilal Aslam	**2022361**
Muhammad Hassaan Shah	**2022385**

Supervisor:
Hafiz Syed Ahmed Qasim

Co-Supervisor:
Muhammad Ahmad Nawaz

Faculty of Computer Science and Engineering

GIK INSTITUTE
OF ENGINEERING SCIENCES AND TECHNOLOGY

Revision History:

Revision History	***Date***	***Comments***
1.00	24/10/2025	Initial draft
2.00	16/11/2025	Diagram improvements + minor changes
3.00	19/11/2025	Adding UI and finalizing report

Document Approval:

The following document has been accepted and approved by the following:

Signature	***Date***	***Name***
	19/11/2025	Hafiz Syed Ahmed Qasim
	19/11/2025	Muhammad Ahmad Nawaz

List of Contents

LIST OF CONTENTS **3**

1. INTRODUCTION **6**

1.1. PURPOSE 7
1.2. PRODUCT SCOPE 7

2. OVERVIEW **8**

2.1. THE OVERALL DESCRIPTION 8
2.2. PRODUCT PERSPECTIVE 8
2.3. PRODUCT FUNCTIONS 9
2.4. USER CHARACTERISTICS 10
2.5. CONSTRAINTS 11
2.6. ASSUMPTIONS AND DEPENDENCIES 12

3. STATE OF THE ART **13**

3.1 LITERATURE REVIEW 13
3.2 EXISTING SYSTEMS 14

4. USER/SYSTEM REQUIREMENTS **15**

4.1. EXTERNAL INTERFACE REQUIREMENTS 15
4.1.1. User Interfaces 15
4.1.2. Hardware Interfaces 16
4.1.3. Software Interfaces 16
4.1.4. Communication Interfaces 17

5. FUNCTIONAL REQUIREMENTS **18**

5.1. FUNCTIONAL REQUIREMENTS WITH TRACEABILITY INFORMATION 18
5.1.1. AI AGENT CONNECTION 18
5.1.2. INTELLIGENT ANALYSIS 19
5.1.3. AUTONOMOUS EXECUTION 19
5.1.4. REAL-TIME ADAPTATION 20
5.1.5. ADVANCED REPORTING 20
5.1.6. PRIVACY AND DATA SECURITY 21

6. NONFUNCTIONAL REQUIREMENTS & SOFTWARE SYSTEM ATTRIBUTES **21**

6.1. PERFORMANCE REQUIREMENTS 21
6.2. NONFUNCTIONAL REQUIREMENTS 22
6.2.1. Reliability & Usability 22
6.2.2. Maintainability 22
6.2.3. Scalability 22
6.2.4. Robustness 22
6.2.5. Testability 23

7. PROJECT DESIGN/ARCHITECTURE **23**

List of Figures

Figure 1: Use Case View.....23
Figure 2: Logical View.....24
Figure 3: Components Diagram.....24
Figure 4: Development View.....25
Figure 5: Context Diagram.....26
Figure 6: Process View.....27
Figure 7: Sequence Diagram for a complete pen-testing process on a target.....27
Figure 8: Activity Diagram.....28
Figure 9: Physical View.....29
Figure 10: AUTOBOTs Local Application UI.....29
Figure 11: Integrated with 5IRE.....30
Figure 12: AUTOBOTs integrated with Cursor.....30

List of Tables

Table 1: Terms used in this document and their description……..07
Table 2: Existing Solutions.....14
Table 3: Functional Requirements 1……..18
Table 4: Functional Requirements 2……19
Table 5: Functional Requirements 3 ……..19
Table 6: Functional Requirements 4……20
Table 7: Functional Requirements 5……20
Table 8: Functional Requirements 6……21

1. INTRODUCTION

In today's rapidly evolving digital landscape, cybersecurity threats are increasing in scale, sophistication, and frequency. Traditional security mechanisms such as periodic vulnerability assessments, manual penetration testing, and reactive remediation are no longer sufficient to protect modern applications operating within dynamic, continuous delivery environments. As organizations accelerate their development cycles, their attack surfaces expand, exposing them to complex and often overlooked vulnerabilities.

When cyber vulnerabilities are exploited, organizations face financial losses, reputational damage, and operational disruption. These risks are often amplified by the slow, error-prone nature of manual testing and the limitations of existing automated tools, which frequently generate false positives and fail to adapt to evolving threats. Traditional methods cannot keep pace with rapid development cycles or the growing sophistication of attacks. This creates a pressing need for AI-driven, autonomous penetration testing capable of continuously and intelligently assessing, validating, and remediating vulnerabilities. AUTOBOTs addresses this need by introducing an adaptive, multi-agent security framework designed to perform real-time, intelligent, and scalable offensive security operations with minimal human intervention.

AUTOBOTs: Autonomous Security Engineer is developed to meet these challenges through an AI-driven, autonomous, and adaptive security testing system. It introduces a multi-agent architecture that leverages Large Language Models (LLMs), vulnerability intelligence, and secure software engineering principles to perform advanced penetration testing, proof-of-concept validation, and automated risk remediation with minimal supervision.

Unlike traditional automation tools, AUTOBOTs employ intelligent decision-making and human-like reasoning to preserve campaign context across multi-step tests, eliminate false positives, and operate at scale across numerous endpoints. By integrating these capabilities into a cohesive framework, the system bridges the gap between offensive security and continuous protection, empowering organizations to maintain compliance with standards such as PCI DSS 4.0, HIPAA, and ISO/IEC 27001:2022, while accelerating secure software delivery.

This document outlines the scope, objectives, technological architecture, and anticipated impact of the AUTOBOTs: Autonomous Security Engineer project, along with the complete system requirements and specifications necessary for its development. As we embark on this initiative, we invite developers, cybersecurity professionals, and industry stakeholders to collaborate in building a future where intelligent, autonomous security systems safeguard digital environments with greater efficiency, accuracy, and resilience. Through this project, we aim to redefine how organizations protect their applications, empowering them to detect, validate, and remediate vulnerabilities proactively and confidently.

1.1. PURPOSE

The primary purpose of AUTOBOTs is to develop an autonomous cybersecurity framework capable of performing, validating, and scaling penetration testing and vulnerability remediation with minimal manual oversight. The system enhances security testing efficiency through AI agents that respond to evolving threats and complex environments while reducing false positives via reproducible proof-of-concept validation and intelligent vulnerability assessment. By preserving context and continuity across extended security campaigns, AUTOBOTs ensures accurate and comprehensive results. It is designed to scale efficiently across large numbers of targets, streamline compliance and remediation processes, and minimize organizational delays while improving accountability. Ultimately, AUTOBOTs aims to redefine cybersecurity practices by shifting from manual, human-dependent testing to an autonomous, intelligent, and continuously adaptive security pattern.

1.2. PRODUCT SCOPE

AUTOBOTs is an AI-driven penetration testing and vulnerability management system designed to autonomously identify, validate, and remediate security flaws across web applications, code-bases, and digital infrastructures. The system operates through a multi-agent architecture where specialized AI agents collaborate to perform reconnaissance, exploit discovery, proof-of-concept validation, and automated patching with minimal human input. By leveraging Large Language Models (LLMs) and vulnerability intelligence, AUTOBOTs delivers context-aware analysis, adaptive testing strategies, and reproducible results to eliminate false positives and improve accuracy.

The product's scope extends to supporting continuous and scalable testing across large enterprise environments, maintaining global context across complex, multi-step campaigns, and generating structured vulnerability reports with validated findings. It will also include mechanisms for compliance tracking with frameworks such as PCI DSS 4.0, HIPAA, and ISO/IEC 27001:2022, helping organizations meet regulatory and audit requirements efficiently.

Table 1: Terms used in this document and their description

Name	Description
API	Application Programming Interface
LLMs	Large Language Models
MCP Gateway	The security guard & coordinator (the middle manager)
Tool Adapter	Executes tool commands safely in a sandbox
HIPAA	Health Insurance Portability and Accountability Act
PCI DSS	Payment Card Industry Data Security Standard

2. OVERVIEW

2.1. THE OVERALL DESCRIPTION

AUTOBOTs is a revolutionary AI-driven web application penetration testing tool that leverages advanced concepts like Modern Context Protocol, multi-agent architecture powered by Large Language Models (LLMs). Its primary goal is to autonomously perform reconnaissance on target web applications, map their attack surfaces, identify and exploit vulnerabilities to assess their potential impact, recommend mitigation strategies, and deliver comprehensive security reports to organizations.

This system integrates autonomous AI agents that have over various reconnaissance and security tools, enabling it to effectively map an organization's digital infrastructure and identify potential entry points that attackers could exploit. By detecting these vulnerabilities before malicious actors can, AUTOBOTs empowers organizations to proactively safeguard their assets, maintain confidentiality, integrity, and privacy, and implement effective mitigation mechanisms well in advance.

Through this proactive and intelligent approach, organizations can significantly reduce financial losses, prevent data breaches, and minimize post-attack recovery costs. AUTOBOTs not only enhances security visibility but also ensures long-term resilience by embedding continuous, autonomous, and intelligent vulnerability assessment into the organization's security workflow.

2.2. PRODUCT PERSPECTIVE

AUTOBOTs is a standalone, web-based AI penetration testing platform designed to integrate seamlessly into an organization's existing security and development infrastructure. It functions as an intelligent offensive security component within the broader cybersecurity ecosystem, complementing tools such as vulnerability scanners and DevSecOps pipelines.

The system operates through a multi-agent architecture, where each agent performs specific security functions such as reconnaissance, exploitation, validation, and reporting under centralized coordination. AUTOBOTs interacts with target web applications, APIs, and databases using secure communication channels and leverages over various tools to gather and analyze data.

AUTOBOTs is designed to provide actionable insights and validated findings that can be directly utilized by security teams for remediation.

AUTOBOTs receives instructions from users and performs OSINT Gathering, Exploit Development, Vulnerability Scanning and much more. It depends on integrated reconnaissance and exploitation tools, external vulnerability databases, and secure API communication. The system requires stable network connectivity and adequate computational resources to support concurrent testing and LLM processing.

2.3. PRODUCT FUNCTIONS

AUTOBOTs performs a series of intelligent, automated cybersecurity functions aimed at identifying, validating, and remediating vulnerabilities across web applications. The core product functions include:

2.3.1. OSINT Gathering

AUTOBOTs automatically collects publicly accessible intelligence, such as exposed domains, technologies, leaked credentials, and infrastructure metadata, to construct an initial attack surface profile without direct interaction. This function enables early-stage reconnaissance and reduces manual effort.

2.3.2. Exploit Development

Using LLM-driven reasoning and vulnerability intelligence feeds, AUTOBOTs can generate, refine, and safely test custom exploit payloads. This capability supports proof-of-concept development and validates real-world exploitability of identified issues.

2.3.3. Vulnerability Scanning

The system autonomously performs comprehensive scans using integrated engines and AI-guided decision-making to detect misconfigurations, outdated software, insecure endpoints, and potential attack vectors. This function enhances coverage and accuracy beyond traditional scanners.

2.3.4. Cloud Security Assessment

AUTOBOTs evaluates cloud environments to identify insecure configurations, exposed services, weak IAM policies, mismanaged roles, and privilege escalation risks. This ensures compliance and strengthens cloud security posture across multi-cloud deployments.

2.3.5. Web Application Testing

The platform conducts advanced dynamic, logic-based, and AI-assisted penetration tests targeting authentication flows, business logic flaws, injections, access control weaknesses, and chained vulnerabilities. It mirrors the techniques used by expert penetration testers.

2.3.6. Target Reconnaissance

AUTOBOTs enumerates endpoints, services, technologies, directories, subdomains, and hidden assets to construct a detailed digital footprint of the target. This identifies exploitable entry points and helps prioritize attack paths.

2.3.7. Bug Bounty Hunting

Through multi-agent reasoning, deep enumeration, and autonomous analysis workflows, AUTOBOTs simulates real bug bounty methodologies. It can uncover high-impact vulnerabilities commonly rewarded in bounty programs.

2.3.8. Authentication Testing

The system evaluates the strength and integrity of authentication components, including session handling, MFA workflows, authorization checks, and privilege escalation behavior. This ensures login mechanisms are secure against common bypasses.

2.3.9. CTF Challenge Solving

AUTOBOTs applies reasoning, sandboxed execution, and tool integration to solve CTF-style puzzles and security challenges. This demonstrates its capacity to analyze unconventional problems and exploit complex security scenarios.

2.4. USER CHARACTERISTICS

The primary users of AUTOBOTs are cybersecurity professionals and development teams responsible for securing web applications and digital infrastructures. These users are expected to have a foundational understanding of web technologies, security testing, and vulnerability assessment principles.

2.4.1. Security Analysts and Penetration Testers

These highly technical users form the primary audience of AUTOBOTs. They are familiar with penetration testing methodologies, exploit development, network scanning, and vulnerability assessment. Their expertise enables them to interpret system-generated findings, validate exploit feasibility, and manage automated assessments. This group benefits most from AUTOBOTs' autonomous multi-agent execution, advanced analysis, and detailed reporting capabilities.

2.4.2. Developers and DevOps Engineers

Developers and DevOps engineers use AUTOBOTs during development, integration, and deployment stages to detect and remediate vulnerabilities early in the SDLC. While not security experts, they possess enough technical knowledge to understand security issues and apply fixes. This group relies on AUTOBOTs for fast, actionable feedback, CI/CD compatibility, and simplified vulnerability explanations.

2.4.3. System Administrators and Security Managers

These users oversee system-wide security posture, compliance adherence, and risk management. They depend on AUTOBOTs for high-level reports, real-time insights, and centralized visibility of organizational vulnerabilities. Their role requires a system that provides reliable reporting, compliance mapping, and continuous monitoring, features that AUTOBOTs is designed to support.

2.5. CONSTRAINTS

The development and operation of AUTOBOTs is subject to several technical, operational, and environmental constraints that influence system performance and scalability.

2.5.1 Network Connectivity

AUTOBOTs requires stable, high-speed network access for reconnaissance, real-time vulnerability scanning, and external intelligence gathering. Unstable connectivity can interrupt agent coordination, delay scanning processes, and degrade system performance. This constraint is critical because many modules rely on cloud APIs and remote tool execution.

2.5.2. Computational Power and Resource Requirements

The system depends on substantial CPU/GPU processing to support multi-agent orchestration, LLM inference, and concurrent penetration tasks. Limited resources can result in slower analysis, reduced AI accuracy, and bottlenecks. Justified because AUTOBOTs is computationally intensive due to its adaptive AI-driven architecture.

2.5.3. Dependence on External Tools and APIs

AUTOBOTs integrates with various scanning engines, OSINT sources, vulnerability feeds, and exploit frameworks. Any service downtime, incompatible API versions, or tool misconfiguration affects system functionality. This constraint exists because AUTOBOTs relies on heterogeneous third-party systems for its core operations.

2.5.4. Data Availability and Quality

LLM reasoning and vulnerability classification depend on high-quality, up-to-date datasets such as CVE feeds, exploit databases, and security intelligence repositories. Poor or outdated datasets can lower detection accuracy. This constraint is justified because AI-driven systems are only as effective as the data supporting them.

2.5.5. Model Training and Maintenance

Maintaining LLM performance requires continuous model fine-tuning, retraining, and updating based on emerging threats. This process demands significant computational and human resources. Without regular model maintenance, AUTOBOTs would lose relevance in a rapidly evolving threat landscape.

2.5.6. Legal and Ethical Constraints

AUTOBOTs must adhere to cybersecurity laws, responsible disclosure policies, and privacy regulations such as GDPR, HIPAA, PCI DSS, and ISO frameworks. Unauthorized or unapproved penetration testing is illegal. This constraint ensures that AUTOBOTs operates within strictly defined ethical and legal boundaries.

2.5.7. Operational Constraints

Effective testing requires privileged access to target systems, secure authentication mechanisms, and encrypted communication channels. Limited permissions can block necessary attack paths. These constraints ensure that the system operates safely, securely, and without disrupting target environments.

2.6. ASSUMPTIONS AND DEPENDENCIES

1. **Assumptions:**

    - Users of AUTOBOTs possess basic technical knowledge of cybersecurity, web application architecture, and vulnerability management to interpret findings and implement recommended remediations.

    - The system will operate in an environment with stable network connectivity and **a**dequate computational resources (CPU, GPU, and memory) to support multi-agent operations and LLMs.

    - Data sources and vulnerability databases used by the system are accurate, up-to-date, and reliable, ensuring valid results during assessments.

- Regular updates, maintenance, and retraining of the AI models will be performed to keep pace with evolving threat landscapes.

2. **Dependencies:**

- AUTOBOTs depends on over various external reconnaissance and security tools, APIs, and frameworks for data collection, vulnerability detection, and proof-of-concept generation.
- It relies on Large Language Models (LLMs) and AI frameworks for intelligent decision-making, contextual analysis, and automation of testing workflows.
- The system is dependent on external vulnerability feeds and intelligence sources such as CVE, NVD, and ExploitDB to ensure accurate and updated results.
- It depends on third-party libraries, APIs, and open-source tools, which must remain compatible and accessible for seamless integration.

3. STATE OF THE ART

3.1 LITERATURE REVIEW

PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing

This paper explores the use of Large Language Models (LLMs) for automating penetration testing and introduces PENTESTGPT, an LLM-based framework that improves task coordination and performance. While effective in executing sub-tasks like tool usage and analysis, PENTESTGPT still faces key limitations: it loses global context during long, multi-step tests, lacks scalability, and requires human oversight. These gaps highlight the need for a more autonomous, context-aware, and scalable solution.

Automated penetration testing: Formalization and realization

This paper proposes ADAPT, an architecture-driven automated penetration testing tool designed to reduce reliance on manual testing by formalizing the penetration testing process at the architectural level. ADAPT uses a self-organizing architecture to automate testing across systems of hosts and services, successfully demonstrating its effectiveness on environments like Metasploitable2 and Metasploitable3. However, the approach lacks real-world exploit validation, relies on predefined vulnerability models, and offers limited capability for detecting or handling zero-day vulnerabilities.

A novel research on design of automated penetration testing system

This paper presents an automated penetration testing system that combines manual and automated methods to assess the security of information systems. The system automates tool execution and improves testing efficiency and accuracy, reducing dependence on specialized human expertise. Experiments on mimicry structure information systems demonstrate its practical applicability and effectiveness. However, the approach relies on pre-defined test strategies, lacks adaptive reasoning and creativity, and stops at detection without validation, leading to potential false positives.

Enhancing Penetration Testing with Machine Learning and Artificial Intelligence: A Comprehensive Analysis

This paper explores the integration of Machine Learning (ML) and Artificial Intelligence (AI) techniques into penetration testing to enhance the speed, accuracy, and effectiveness of security assessments. By automating vulnerability discovery, predicting attack vectors, and simulating complex attack scenarios, the study highlights how AI-driven testing can proactively strengthen organizational cybersecurity. However, the research primarily focuses on specific web vulnerabilities, operates on a limited scale, and lacks integration with CI/CD pipelines, resulting in a slow testing cadence that restricts continuous and large-scale application.

3.2 EXISTING SYSTEMS

Application Name	Features	Weaknesses	How These Weaknesses Are Addressed in AUTOBOTs
XBOW	Autonomous vulnerability scanning Multi-layer attack graph analysis Security posture monitoring	No exploit execution Limited business logic testing No adaptive AI reasoning	Adds LLM-driven reasoning Performs real exploit simulation Supports business logic testing Uses adaptive multi-agent workflows
Nua Security	AI-assisted code scanning SaaS security threat detection Automated policy and compliance checks	Focused only on SaaS applications Limited exploit verification No multi-agent penetration testing	Expands to full-stack penetration testing Adds exploit validation and PoC checks Enables multi-agent orchestration and endpoint assessment

Project Naptime	Automated reconnaissance OSINT gathering High-speed enumeration	Recon-only tool No vulnerability validation No exploitation or remediation capabilities	Full lifecycle workflow: recon, vulnerability detection, exploitation, remediation Validated PoC execution Automated remediation recommendations
Gecko Security	AI code review Vulnerability insights Developer-focused feedback	Limited public documentation No exploit simulation No automated patch validation	Adds autonomous recon Generates exploits and PoCs Validates patches automatically Provides advanced remediation steps
PentestGPT	LLM-based penetration testing guidance Step-by-step reasoning Supports tool execution	Poor long-context retention No real exploit validation Limited scalability Requires human guidance	Multi-agent autonomous architecture Long-term context memory engine Automatic exploit validation Scalable endpoint-wide testing

Table 2: Existing Solutions

4. USER/SYSTEM REQUIREMENTS

4.1. External Interface Requirements

4.1.1. User Interfaces

The AUTOBOTs platform provides a secure, intuitive, and accessible user interface designed to support real-time penetration testing workflows. The system is built with usability, responsiveness, and seamless integration in mind, enabling users to operate the platform without requiring extensive cybersecurity expertise. AUTOBOTs supports IDE integration, allowing developers and security engineers to invoke autonomous testing directly within their development environment. The system can be connected to modern AI-powered IDEs such as Cursor, Roo Code, and Visual Studio Code, enabling inline vulnerability detection, real-time exploit validation, and code-level remediation guidance.

The platform also supports integration with cross-platform desktop AI assistants, like 5IRE, where users can select different large language models, such as Claude, GPT, or GitHub Copilot, to enhance reasoning, analysis, and decision-making throughout the security testing process. Through these interfaces, AUTOBOTs ensures a cohesive, accessible, and secure user experience across diverse technical environments, enabling users to perform autonomous, high-fidelity penetration testing with minimal friction.

4.1.2. Hardware Interfaces

The AUTOBOTs: Autonomous Security Engineer system is a software-based application and does not directly interface with or depend on any specialized hardware components. It operates entirely within standard computing environments using existing hardware resources such as servers, personal computers, or cloud infrastructure. The system requires only basic hardware capabilities to support its computational and networking needs.

4.1.3. Software Interfaces

The AUTOBOTs: Autonomous Security Engineer system interacts with several software components and external services to perform autonomous penetration testing, vulnerability validation, and reporting.

- **Operating Systems:**
  AUTOBOTs is compatible with Linux, Windows Server, and macOS Monterey and later. It can also be deployed in cloud environments such as AWS, Azure, or Google Cloud.

- **External Tools and Libraries:**
  AUTOBOTs integrates with various open-source reconnaissance and exploitation tools (e.g., Nmap, Nikto, Metasploit, Gobuster, and Burp Suite) through command-line execution and API-based communication. It also utilizes Python libraries for network interaction, AI inference, and LLM operations.

- **AI and LLM Frameworks:**
  The system can also use fine-tuned Large Language Models (LLMs) via OpenAI API and Hugging Face Transformers to enable reasoning, context preservation, and autonomous decision-making during penetration testing workflows.

- **Shared Data and Services:**
  All agents and modules share data through an internal API and a centralized database to ensure synchronization and contextual consistency. Inter-process communication (IPC) is implemented via RESTful APIs, ensuring modular scalability. No global data areas are used to prevent security risks.

Data Flow Overview

The following describes the logical data flow between AUTOBOTs' major software interfaces:

1. **User Input / API Trigger:**
   Users or external systems initiate a test campaign through the web UI or REST API.
2. **Task Orchestration (AI Core):**
   The AI controller receives configurations and coordinates security tools via the OS interface.
3. **Tool Execution and Data Collection:**
   External tools perform reconnaissance and vulnerability scanning. Their outputs are fetched, parsed, and standardized.
4. **AI Reasoning and Validation:**
   The processed data is passed to the LLM module for contextual interpretation, exploit suggestion, or validation of findings.
5. **Threat Intelligence Enrichment:**
   Identified vulnerabilities are cross-referenced with external feeds (CVE/NVD/ExploitDB) to determine severity and exploitability.

4.1.4. Communication Interfaces

The AUTOBOTs platform relies on secure and standardized communication protocols to facilitate data exchange between system components, external tools, and users.

- **Web Communication:**
  The user interface is web-based and accessible through modern browsers (e.g., Chrome, Firefox, Edge) using the HTTPS protocol for secure interaction. Communication between the client and server is encrypted with TLS 1.3 to ensure data confidentiality and integrity.

- **Network Protocols:**
  The system communicates with target applications and external tools using standard network protocols such as HTTP/HTTPS, SSH, FTP, DNS, and TCP/IP. These are used during reconnaissance, exploitation, and vulnerability validation.

- **APIs and External Services:**
  AUTOBOTs integrates with external vulnerability databases (CVE, NVD, ExploitDB) and ticketing systems (Jira, ServiceNow) via RESTful APIs. JSON is used as the primary message format for interoperability.

- **Email and Notification Services:**
  The platform supports SMTP-based email notifications to alert users about completed tests, detected vulnerabilities, or system anomalies.

- **Data Transfer and Synchronization:**
  Data exchange between agents, the controller, and the reporting module occurs asynchronously via secured REST endpoints, ensuring real-time synchronization and minimal latency.

5. Functional Requirements

- AI Agent Connection
- Intelligent Analysis
- Autonomous Execution
- Real-time Adaptation
- Database Integration
- Advanced Reporting
- Privacy and Data Security

5.1. Functional Requirements with Traceability information

5.1.1. AI Agent Connection

Requirement ID	FR-1		**Requirement Type**		Functional		**Use Case #**		UC-1
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	PR-1								
Description	The system shall establish secure communication between multiple AI agents and the central controller to enable coordination, task assignment, and data sharing.								
Rationale	Enables distributed, multi-agent collaboration essential for autonomous security testing.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	Agents connect successfully, exchange task data, and remain synchronized without communication failures.								
Dependencies	Stable network connection, API endpoints, and authentication keys.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	Initial version								

Table 3: Functional Requirements 1

5.1.2. Intelligent Analysis

Requirement ID	FR-2		**Requirement Type**		Functional		**Use Case #**		UC-2
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	PR-2								
Description	The system shall use LLM to gather & analyze collected data, interpret vulnerabilities, and determine optimal next steps.								
Rationale	Provides human-like reasoning to enhance detection accuracy and reduce false positives.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	AI correctly interprets scan outputs and recommends appropriate actions with >85% accuracy.								
Dependencies	LLM API, training datasets in case of fine-tuning LLM, and computational resources.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	Initial Version								

Table 4: Functional Requirements 2

5.1.3. Autonomous Execution

Requirement ID	FR-3		**Requirement Type**		Functional		**Use Case #**		UC-3
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	PR-3								
Description	The system shall autonomously execute penetration testing tasks such as reconnaissance, exploitation, and validation with minimal manual supervision.								
Rationale	Enables continuous, scalable testing with minimal human input.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	System completes full testing workflows independently and logs all actions.								
Dependencies	Connected AI agents, integrated tools, and permissioned targets.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	Initial Version								

Table 5: Functional Requirements 3

5.1.4. Real-time Adaptation

Requirement ID	FR-4		**Requirement Type**		Functional		**Use Case #**		UC-4
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	PR-4								
Description	The system shall dynamically adapt testing strategies based on intermediate results and environmental feedback in real time.								
Rationale	Ensures efficiency and adaptability against changing system responses or defenses.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	System modifies test flow automatically upon detecting new security behaviors.								
Dependencies	AI inference engine and continuous feedback loop.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	Initial Version								

Table 6: Functional Requirements 4

5.1.5. Advanced Reporting

Requirement ID	FR-5		**Requirement Type**		Functional		**Use Case #**		UC-5
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	PR-5								
Description	The system shall generate comprehensive, customizable reports detailing vulnerabilities, exploitation results, and mitigation recommendations.								
Rationale	Provides actionable insights and documentation for security teams and compliance audits.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	Reports generated in multiple formats (PDF, DOCX, HTML) with validated findings.								
Dependencies	Reporting engine.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	Initial Version								

Table 7: Functional Requirements 5

5.1.6. Privacy and Data Security

Requirement ID	FR-6		**Requirement Type**		Functional		**Use Case #**		UC-6
Status	***New***	X	***Agreed-to***	-	***Baselined***	-	***Rejected***	-	
Parent Requirement #	PR-6								
Description	The system shall ensure data privacy and security through secure use of APIs and keeping the session stateless.								
Rationale	Protects sensitive test data and complies with industry standards such as PCI DSS and ISO 27001.								
Source	-				**Source Document**		-		
Acceptance/Fit Criteria	All data are will be stored in local file system temporarily and will be deleted after every session closes to ensure data confidentiality.								
Dependencies	Use of secure APIs.								
Priority	***Essential***	X	***Conditional***	-	***Optional***	-			
Change History	Initial Version								

Table 8: Functional Requirements 6

6. Nonfunctional Requirements & Software System Attributes

This section defines the quality attributes necessary to ensure that the system operates reliably, securely, and efficiently. These attributes support both customer expectations and developer needs, covering dimensions such as performance, usability, maintainability, security, and interoperability.

6.1. Performance Requirements

- **Response Time**
  The system shall process and return AI-generated analyses or decisions within 5–8 seconds under normal load.

- **Execution Throughput**
  The system shall handle at least 10 concurrent penetration-testing tasks without degradation of system performance.

- **Real-Time Adaptation Speed**
  The system shall update its testing strategy within 2 seconds of receiving new environmental feedback or tool output.

- **Resource Utilization**
  AI agents and supporting modules shall not exceed 70% CPU utilization and 80% memory utilization under typical workloads.

- **Scalability**
  The system must support horizontal scaling, enabling deployment of additional AI agents without requiring architectural changes.

6.2. Nonfunctional Requirements

Nonfunctional requirements describe the quality attributes and operational conditions of the system, defining how it should perform rather than what it does. They cover aspects such as performance, security, usability, reliability, and scalability to ensure the system operates efficiently, securely, and consistently.

6.2.1. Reliability & Usability

- The system shall maintain 99% uptime excluding scheduled maintenance.
- AI agents must recover automatically from failures and resume task execution.

- The user interface shall follow a clean, intuitive layout requiring <1 hour for a new user to learn basic operations.
- Navigation shall not require more than 3 clicks to reach any primary function.

6.2.2. Maintainability

- The system architecture shall support modular updates where AI agents, tools, or data pipelines can be replaced without impacting other components.
- Codebase shall follow standard documentation practices, targeting a mean time to repair (MTTR) of <2 hours for minor faults.

6.2.3. Scalability

- The system shall support dynamic expansion of AI agents and compute nodes without downtime.
- The architecture must support cloud-based autoscaling where resource allocation adjusts based on demand.

6.2.4. Robustness

- The system must handle unexpected or malformed tool outputs without failure.
- Error handling processes must ensure that system crashes do not interrupt ongoing tests, preserving all logs.

6.2.5. Testability

- Each subsystem (AI agent, database, reporting, execution engine) shall support automated unit and integration testing with ≥90% coverage.
- Logs must include detailed traces for all system actions to support debugging and auditing.

7. Project Design/Architecture

- 4+1 ARCHITECTURE VIEW MODEL
  - Use Case View
  - Logical View:
  - Development View
  - Process View
  - Physical View
  - User Interface Design

7.1. Use Case View:

Provides a high-level representation of how different user groups (analysts, developers, administrators) interact with AUTOBOTs to perform autonomous security testing tasks.

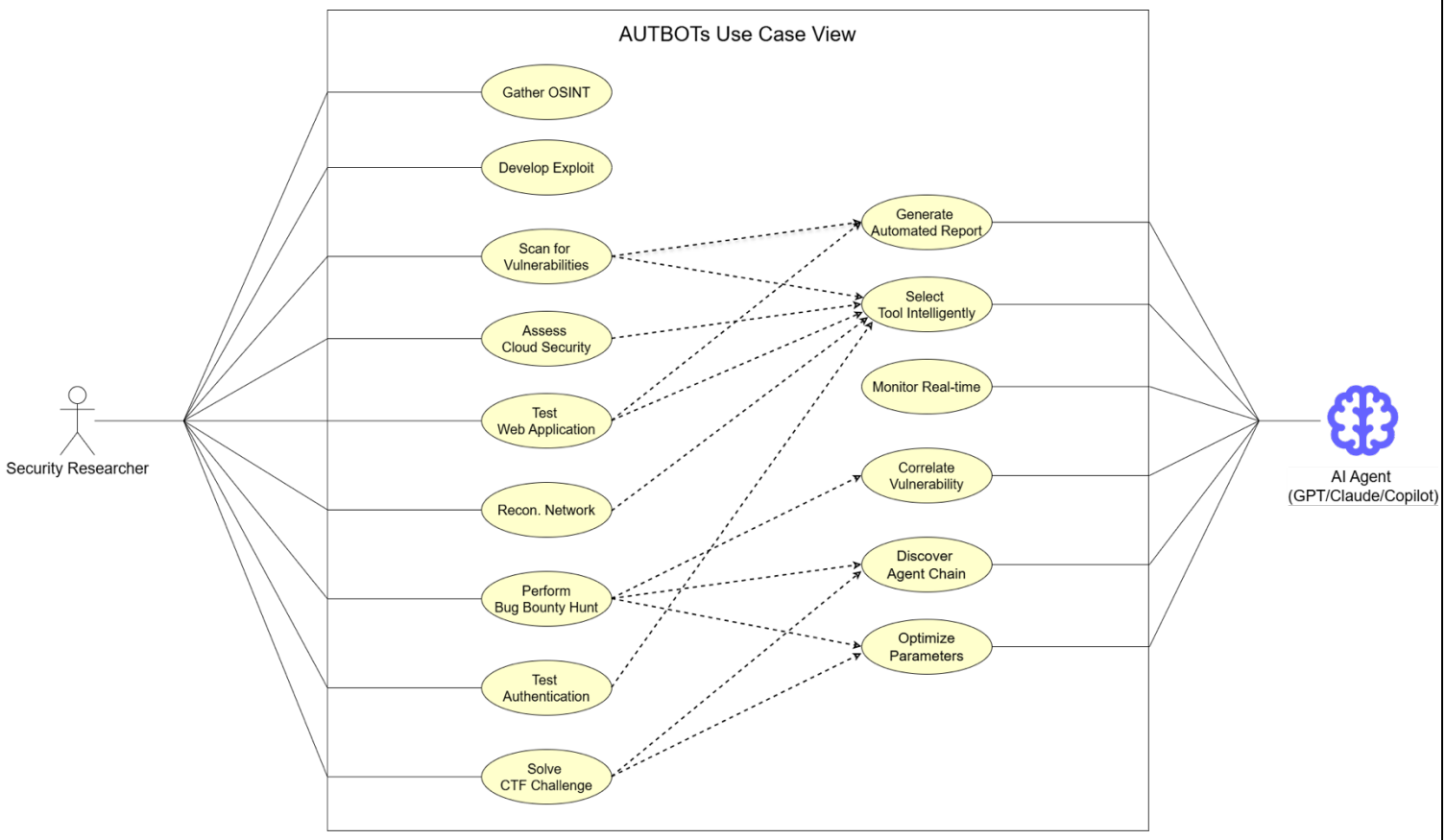

Figure 1: Use Case View

7.2. Logical View:

Defines the system's functional structure, showing how modules like AI agents, scanning engines, orchestration logic, and reporting components collaborate.

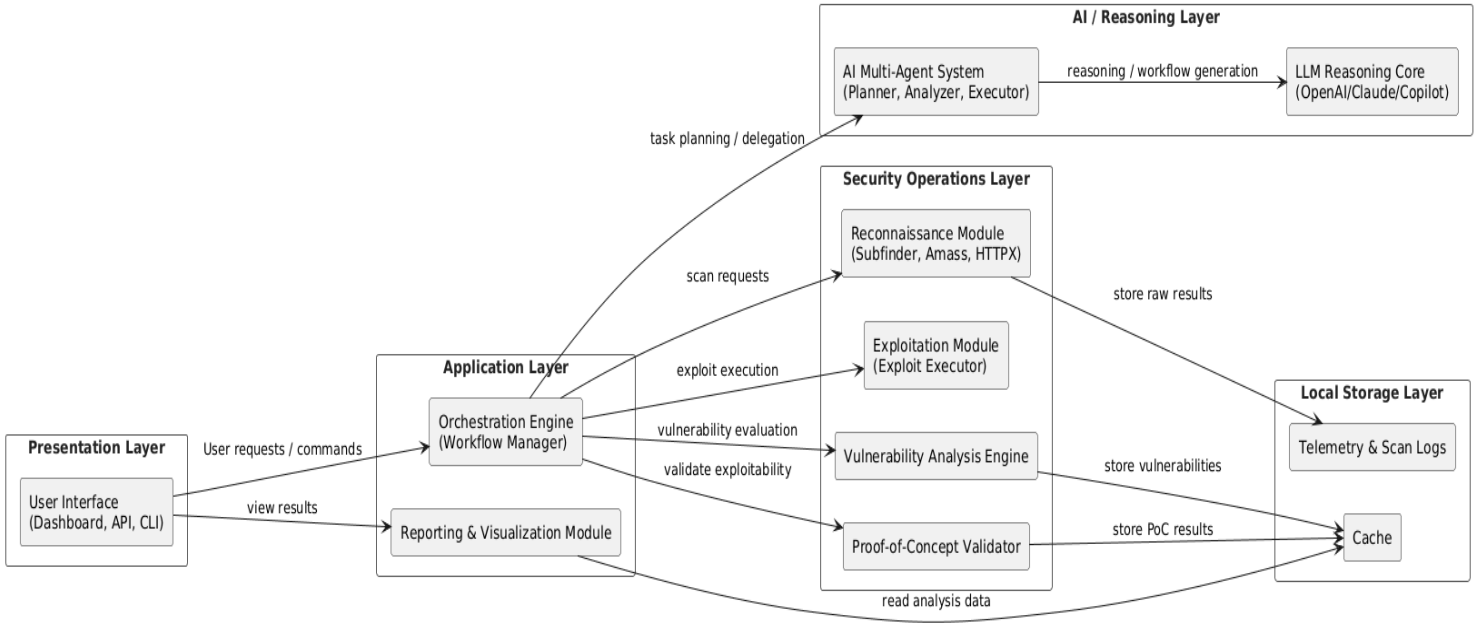

Figure 2: Logical View

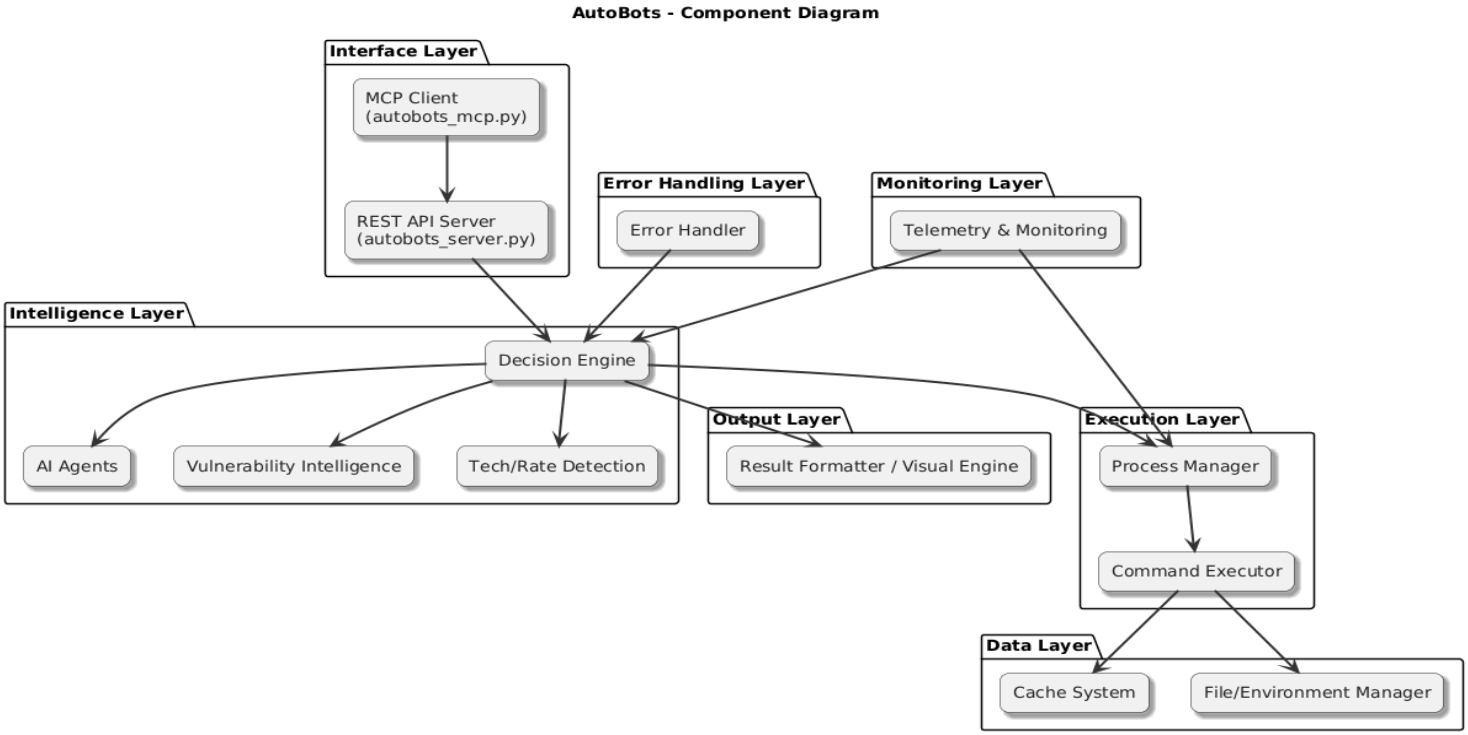

Figure 3: Components Diagram

7.3. Development View

Describes the internal software organization, including layered architecture, modules, code components, and their responsibilities during implementation.

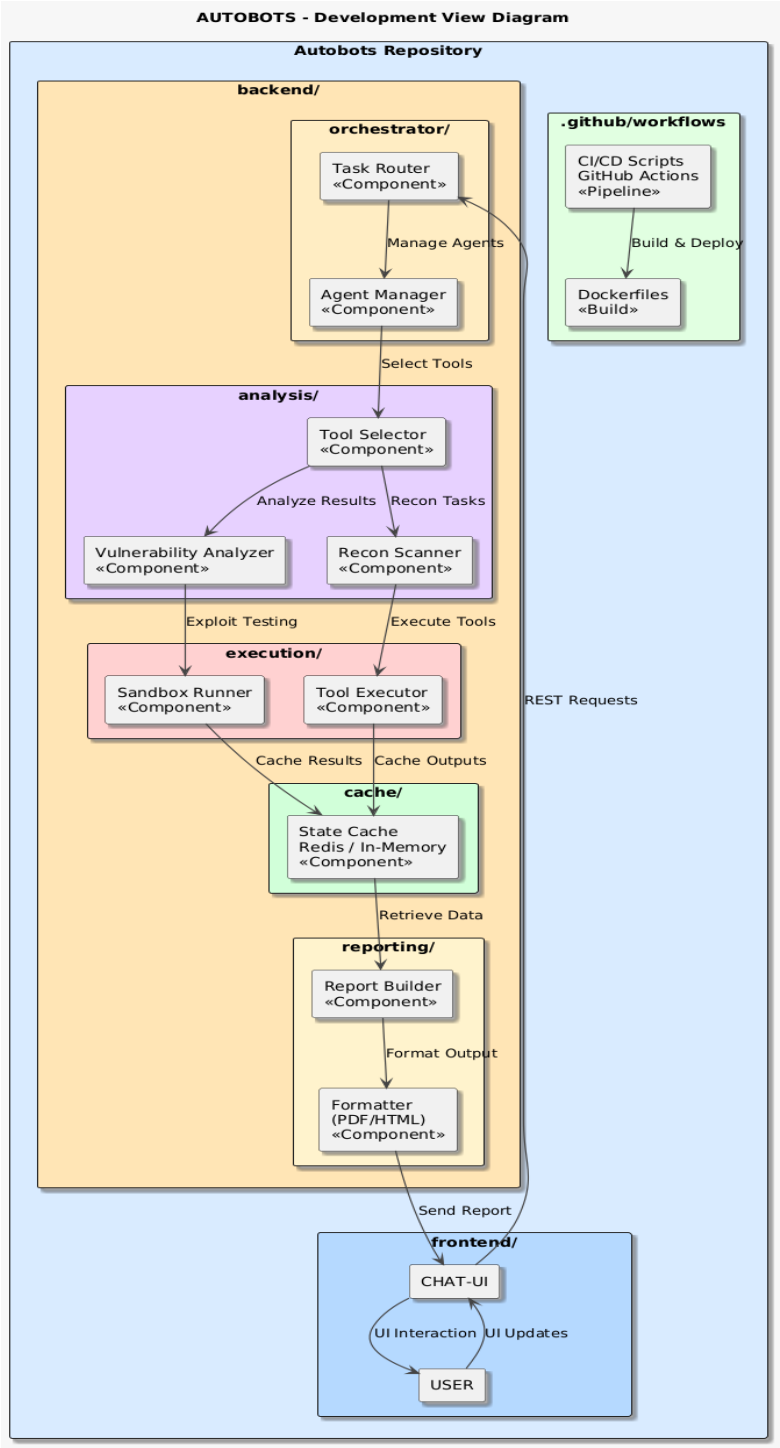

Figure 4: Development View

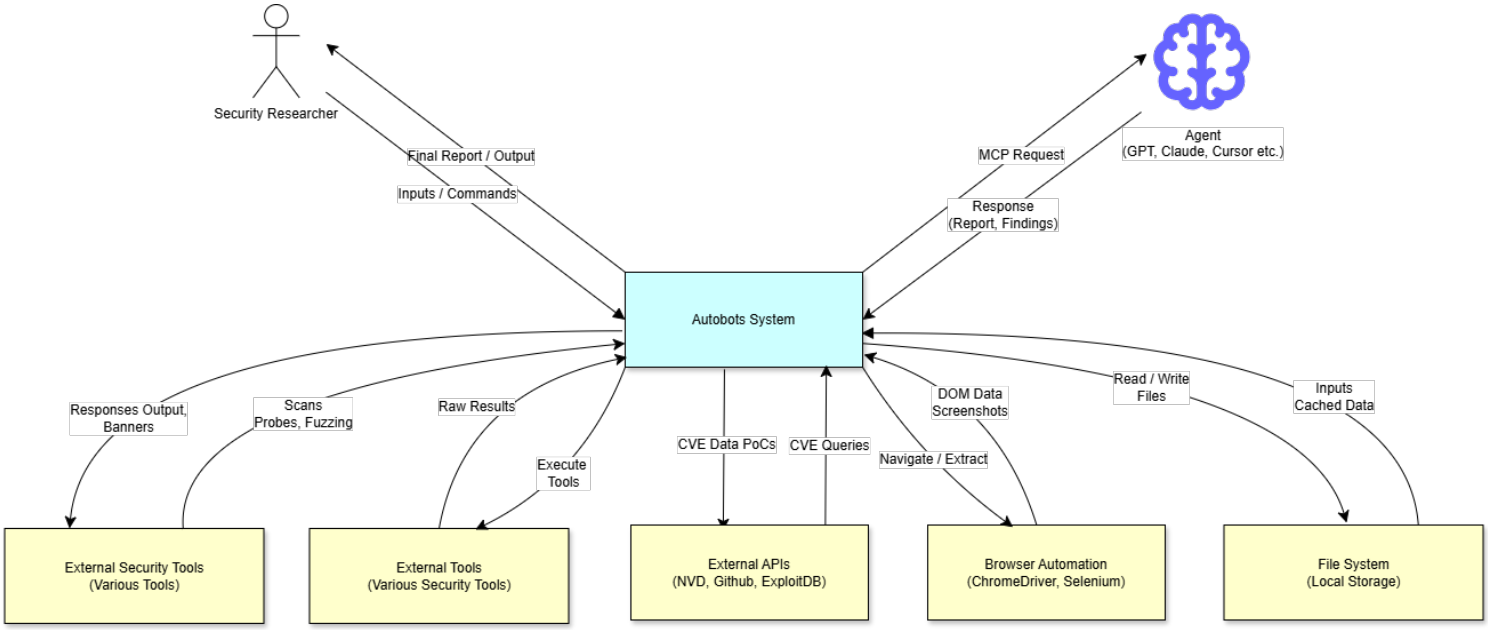

Figure 5: Context Diagram

7.4. Process View

Explains the runtime behavior of AUTOBOTs, detailing workflows, agent interactions, task orchestration, concurrency, and data flow during autonomous penetration testing.

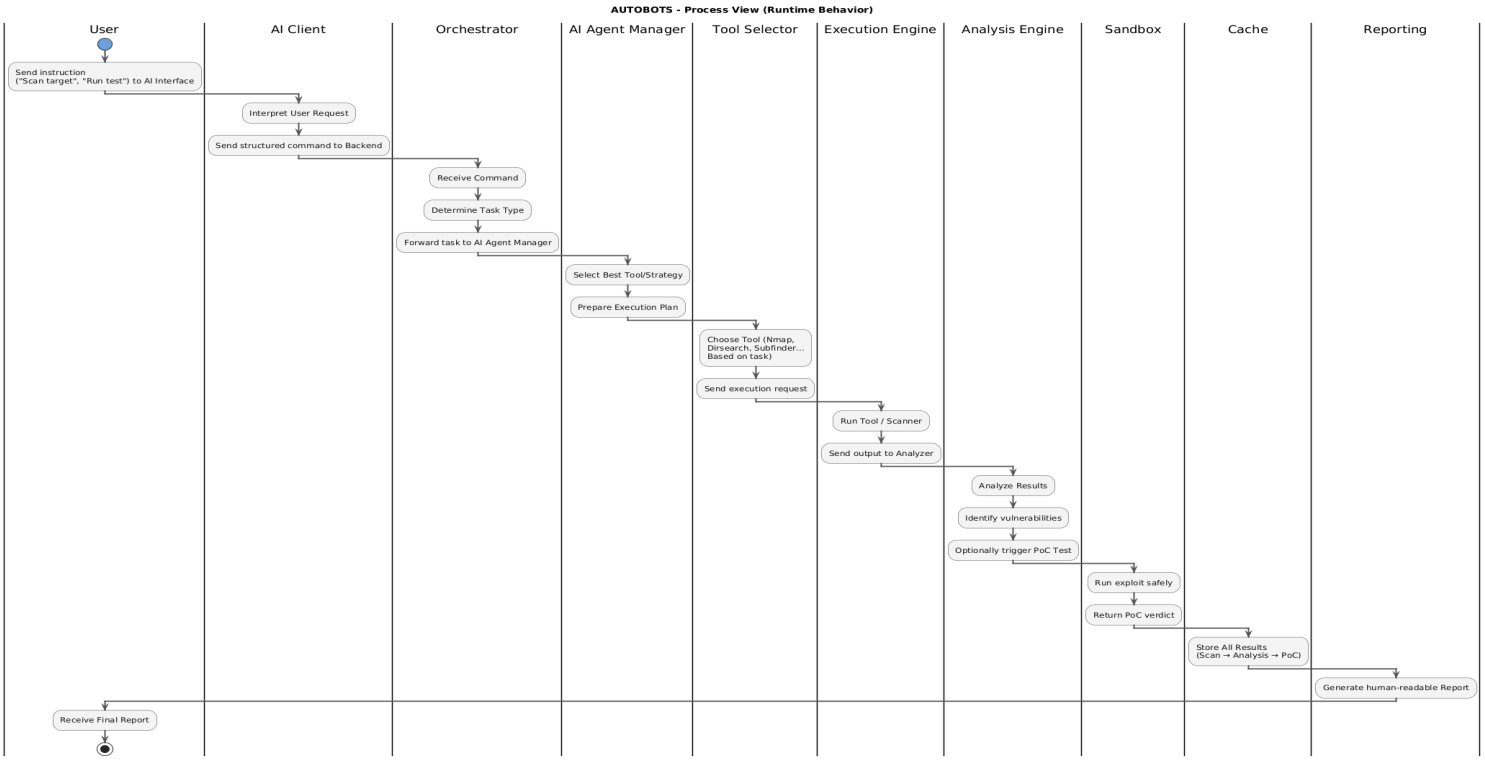

Figure 6: Process View

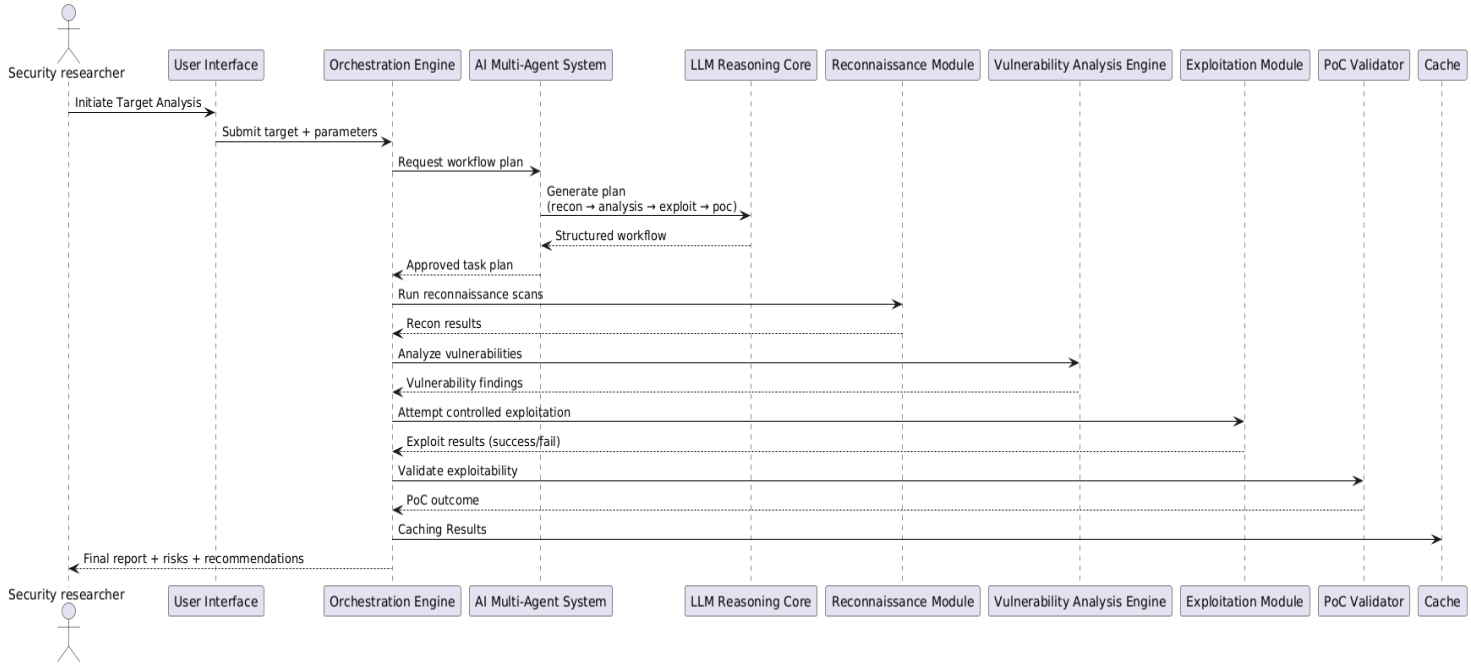

Figure 7: Sequence Diagram for a complete pen-testing process on a target

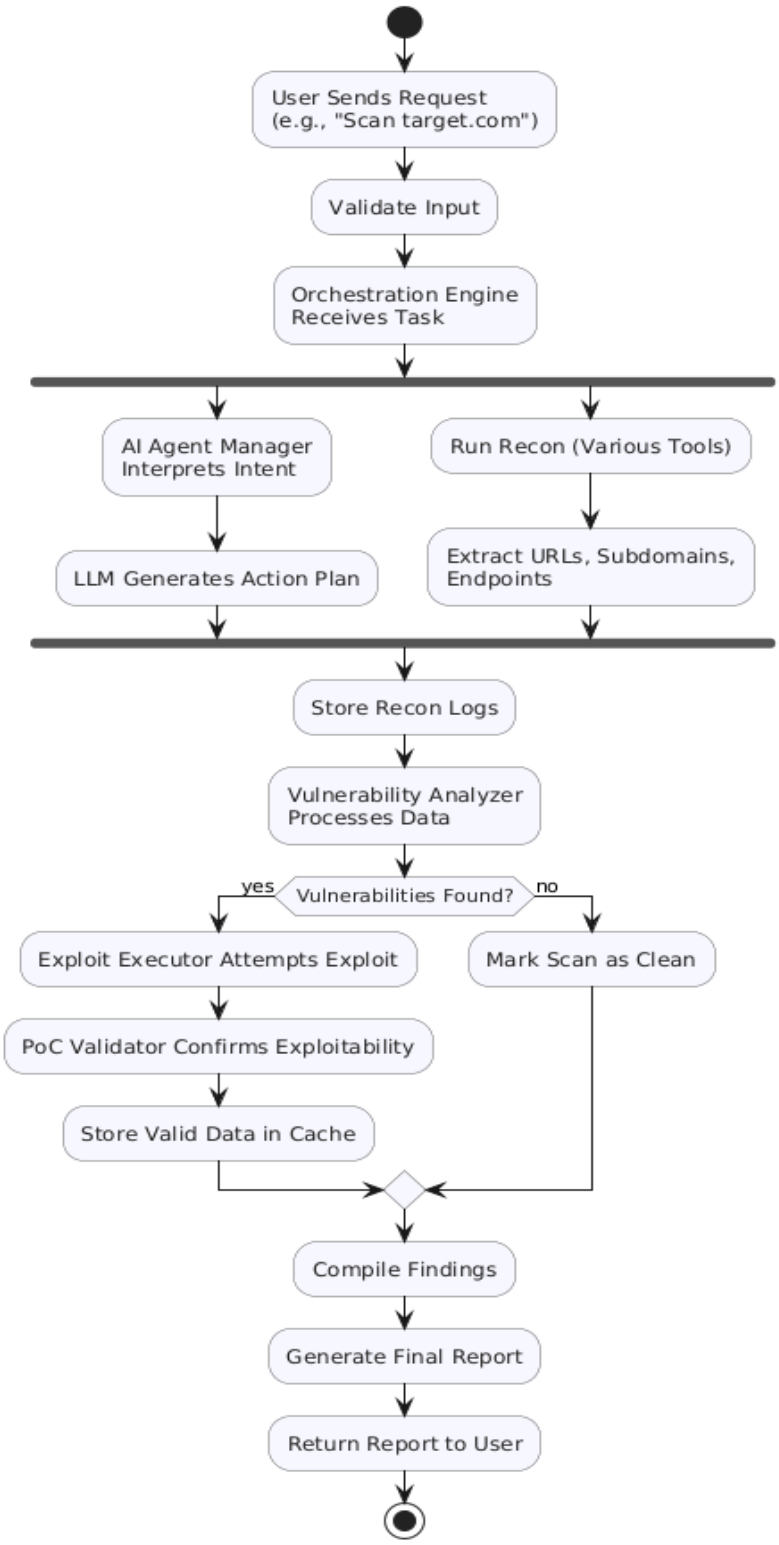

Figure 8: Activity Diagram

7.5. Physical View

Illustrates how AUTOBOTs' components are deployed across client and server environments in the two-tier architecture, including resource allocation and network communication.

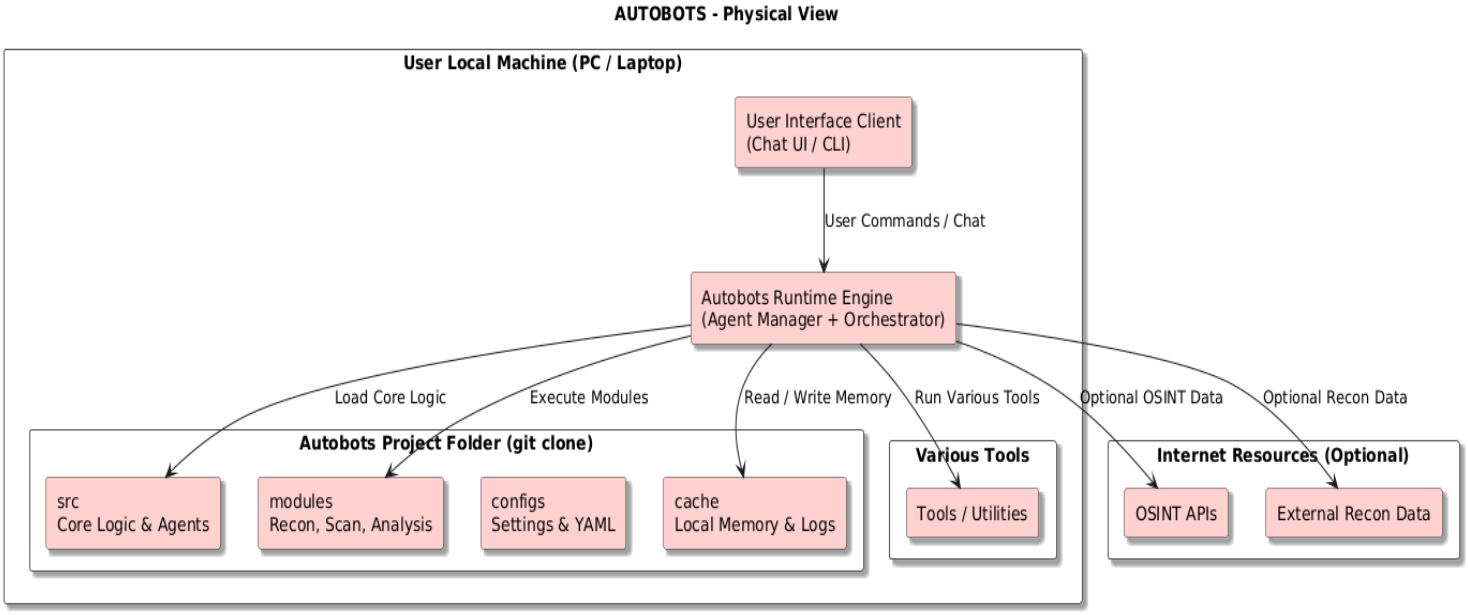

Figure 9: Physical View

7.6. User Interface Design

Figure 10: AUTOBOTs Local Application UI

Along with this, we also aim to introduce seamless integration in security researchers' day-to-day solutions e.g.:

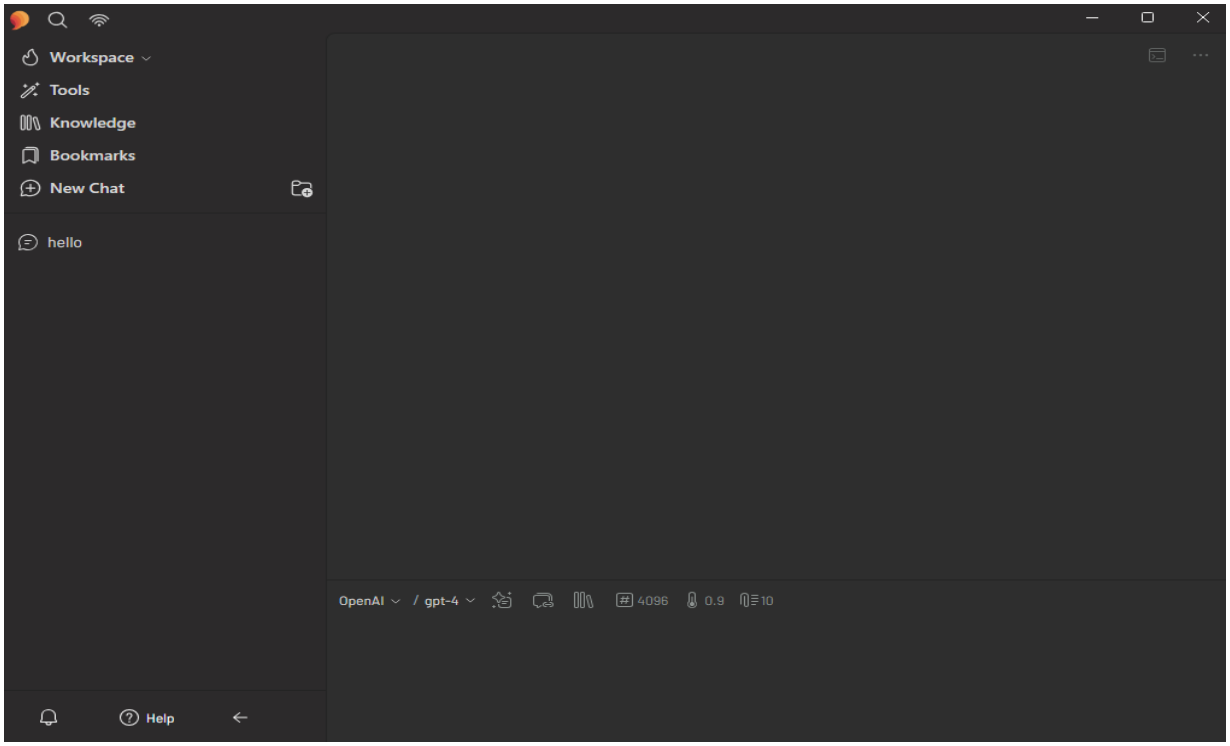

Figure 11: Integrated with 5IRE

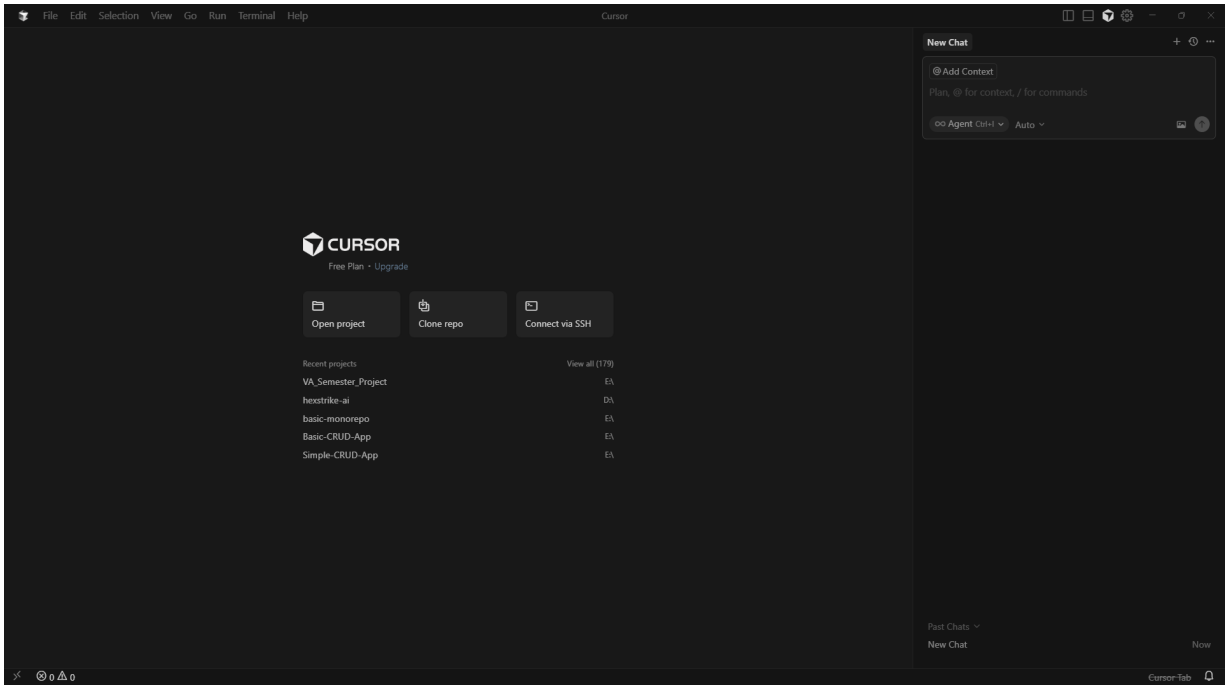

Figure 12: AUTOBOTs integrated with Cursor